

28A – M8 Reporting Pipeline: Implementation Guide

Everything the M8 developer needs: the reporting schema as DDL, the projection consumer pattern with the rebuild path, freshness computation, end-of-day, the report definition format and the phase-1 catalogue as data, the renderer and export workers, the analyst role setup, dashboards, screens, tests and tickets. Self-contained.

Field	Value
Document	28A – implementation guide for 28 M8 Reporting Pipeline v0.1
Version	v0.1 – for the M8 developer
Date	9 September 2026
Prerequisite	Sprint 0 (event backbone with archive and replay, A4 renderer, scheduler); the event contracts of docs 21A–27A (payload shapes); the replica in place (doc 35 for production; compose profile for development)
Estimated effort	About 18 developer-days for one full-stack developer, plus the definitions that arrive from the workshop as data
Restated content	Marked <i>restated from doc X §Y</i> .

Read this first. Every table in the reporting schema must be droppable: build the projection consumers so that `RebuildProjection(name, 0)` produces the same rows as the live consumer did, and make the equivalence test the first test you write. If a projection cannot be rebuilt from the archive, it is not a projection; it is a second source of truth, and it does not belong here.

1. What you are building

Restated from doc 28 §1. The pipeline: projections, freshness, end-of-day, the catalogue as data, rendering and export, the analyst role, dashboards and exception queues. Not the reports themselves.

You build	You do not build
Schema reporting: thirteen projections, location_freshness, report_definition, report_run, export_job, projection_state, dashboard_tile, exception_item	Any figure that is not derived from events; any write to another schema
Thirteen projection consumers with a shared upsert-and-rebuild pattern; the freshness consumer; end-of-day and roll-up jobs; the render and export workers; the analyst role setup	The audit store (kernel); acknowledgement of exceptions (kernel command)
Seven web screens; the phase-1 definitions as seed data (twelve reports)	The workshop's definitions (they arrive as data in the same format)

2. What you inherit

From	You use	How
19A §5 event backbone	@EventConsumer with inbox; Replayer.replay(consumer, fromSeq); the archive partitions	Every projection; rebuilds
19A §6 A4Renderer	Templates and rendering worker; three-language fixed strings	Report rendering
19A §12 scheduler, §13 business date	@ScheduledJob; location.day_closed.v1	End-of-day, roll-ups
19A §1 scope	RLS classes on projections; FEDERATION_VIEW session for the analyst role	Section 3, 7.4
19A §4 audit	ExceptionQueueQuery semantics; AUDIT_REVIEWED events	Exception items projection
Docs 21A–27A	Event payloads: receipt bundles (26A §3), stock.moved (25A), M4 events (24A §5.3), M7 events (27A), M1 device/location events (21A)	Consumer inputs
M2 query	ResolveAlias; names for rendering	Canonical SKU; labels
17A §7 web shell	FreshnessStamp, ExceptionQueue, tables, generated client	Section 8

3. Schema

Restated from doc 28 §3 (CR-28-1). Location db/migration/m8reporting/. Every table carries owner_entity_id and, where meaningful, location_id, with the four RLS classes; PARTY-readable tables have no cost or margin columns.

```
-- V0001_reporting.sql (excerpt; RLS per 17A 6.3 on every table)
CREATE TABLE reporting.projection_state (name text PRIMARY KEY, consumer text NOT NULL, last_seq bigint NOT
  NULL DEFAULT 0, last_event_at timestamptz,
  status text NOT NULL DEFAULT 'LIVE' CHECK (status IN ('LIVE', 'REBUILDING', 'LAGGING')), rebuild_started_at
  timestamptz, shadow_table text);

CREATE TABLE reporting.location_freshness (location_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL,
  applied_through timestamptz NOT NULL,
  last_batch_at timestamptz NOT NULL, devices_reporting smallint NOT NULL DEFAULT 0);

CREATE TABLE reporting.receipt_fact (receipt_id uuid NOT NULL, received_at timestamptz NOT NULL,
  owner_entity_id uuid NOT NULL, location_id uuid NOT NULL,
  till_position_id uuid, business_date date NOT NULL, operator_user_id uuid, status text NOT NULL, gross
  numeric(14,2), tax numeric(14,2),
  bill_discount numeric(14,2), rounding numeric(14,2), tender_cash numeric(14,2), tender_card numeric(14,2),
  tender_account numeric(14,2),
  lines smallint, engine_version text, snapshot_version bigint, voided_by uuid, PRIMARY KEY (received_at,
  receipt_id)) PARTITION BY RANGE (received_at);
CREATE TABLE reporting.receipt_line_fact (line_id uuid NOT NULL, received_at timestamptz NOT NULL, receipt_id
  uuid NOT NULL, owner_entity_id uuid NOT NULL,
  location_id uuid NOT NULL, business_date date NOT NULL, sku_id uuid NOT NULL, canonical_sku_id uuid NOT
  NULL, batch_id uuid, uom_code varchar(10),
  qty numeric(14,3), unit_price numeric(14,4), mrp_applied numeric(14,2), control_price_applied numeric(14,2),
  cap_reason text, rule_id uuid,
  rule_kind text, discount numeric(14,2), tax_rate numeric(5,2), tax numeric(14,2), line_total numeric(14,2),
  unit_cost numeric(14,4), -- cost: OWN/FED only
  negative_ack boolean, picked boolean, PRIMARY KEY (received_at, line_id)) PARTITION BY RANGE (received_at);
CREATE INDEX rlf_daily ON reporting.receipt_line_fact (location_id, business_date, canonical_sku_id);

CREATE TABLE reporting.sales_daily (owner_entity_id uuid NOT NULL, location_id uuid NOT NULL, business_date
  date NOT NULL, canonical_sku_id uuid NOT NULL,
  qty numeric(14,3) NOT NULL, net numeric(14,2) NOT NULL, tax numeric(14,2) NOT NULL, discount numeric(14,2)
  NOT NULL, markdown numeric(14,2) NOT NULL,
  cogs numeric(14,2), receipts int NOT NULL, freshness timestamptz NOT NULL, PRIMARY KEY (location_id,
  business_date, canonical_sku_id));
CREATE TABLE reporting.cash_daily (location_id uuid NOT NULL, business_date date NOT NULL, owner_entity_id
  uuid NOT NULL, sessions smallint, float_total numeric(14,2),
  cash_tenders numeric(14,2), change_given numeric(14,2), drops numeric(14,2), counted numeric(14,2), variance
  numeric(14,2), banked numeric(14,2),
  freshness timestamptz NOT NULL, PRIMARY KEY (location_id, business_date));
CREATE TABLE reporting.stock_position (location_id uuid NOT NULL, batch_id uuid NOT NULL, condition text NOT
  NULL, owner_entity_id uuid NOT NULL,
  canonical_sku_id uuid NOT NULL, qty_on_hand numeric(14,3) NOT NULL, unit_cost numeric(14,4), expiry_date
  date, received_at timestamptz,
  days_held int, negative_since timestamptz, freshness timestamptz NOT NULL, PRIMARY KEY (location_id,
  batch_id, condition));
CREATE TABLE reporting.stock_movement_fact (movement_id uuid NOT NULL, received_at timestamptz NOT NULL,
  owner_entity_id uuid NOT NULL, location_id uuid NOT NULL,
  canonical_sku_id uuid NOT NULL, batch_id uuid, movement_type text NOT NULL, qty_delta numeric(14,3),
  unit_cost numeric(14,4), document_id uuid,
  occurred_at timestamptz, PRIMARY KEY (received_at, movement_id)) PARTITION BY RANGE (received_at);
CREATE TABLE reporting.loss_fact (loss_id uuid NOT NULL, received_at timestamptz NOT NULL, owner_entity_id
  uuid NOT NULL, location_id uuid NOT NULL,
  business_date date NOT NULL, canonical_sku_id uuid NOT NULL, batch_id uuid, category text NOT NULL, qty
  numeric(14,3), value numeric(14,2),
  document_id uuid, approver uuid, source text NOT NULL CHECK (source IN
  ('WRITEOFF', 'COUNT_ADJUST', 'NEGATIVE_CLEAR')), PRIMARY KEY (received_at, loss_id)) PARTITION BY RANGE
  (received_at);
CREATE TABLE reporting.trade_document_fact (document_id uuid PRIMARY KEY, doc_type text NOT NULL,
  seller_entity_id uuid, buyer_entity_id uuid, relationship_id uuid,
  owner_entity_id uuid NOT NULL, business_date date, status text, net numeric(14,2), tax numeric(14,2), gross
  numeric(14,2), settled numeric(14,2), credited numeric(14,2),
  committed_eta date, delivered_at timestamptz, fill_rate numeric(5,2), reference_document_id uuid, updated_at
  timestamptz NOT NULL);
CREATE TABLE reporting.dispute_register (document_id uuid PRIMARY KEY, kind text, seller_entity_id uuid,
  buyer_entity_id uuid, owner_entity_id uuid NOT NULL,
  raised_at timestamptz, window_ends_at timestamptz, status text, resolution text, escalated_at timestamptz,
  arbitrated boolean, value numeric(14,2));
CREATE TABLE reporting.exposure_snapshot (relationship_id uuid NOT NULL, as_of date NOT NULL, owner_entity_id
  uuid NOT NULL, amount numeric(14,2), open_invoices numeric(14,2),
  accepted_not_invoiced numeric(14,2), unapplied_credits numeric(14,2), unapplied_receipts numeric(14,2),
```

```

        credit_limit numeric(14,2), freshness timestampz, PRIMARY KEY (relationship_id, as_of));
CREATE TABLE reporting.customer_receivables (account_id uuid NOT NULL, as_of date NOT NULL, owner_entity_id
    uuid NOT NULL, balance numeric(14,2),
    b0_30 numeric(14,2), b31_60 numeric(14,2), b61_90 numeric(14,2), b90p numeric(14,2), PRIMARY KEY
    (account_id, as_of)); -- no names or phones
CREATE TABLE reporting.till_session_fact (session_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL,
    location_id uuid NOT NULL, till_position_id uuid, business_date date,
    operator_user_id uuid, opened_at timestampz, closed_at timestampz, float_amount numeric(14,2),
    cash_tenders numeric(14,2), drops numeric(14,2),
    counted numeric(14,2), expected numeric(14,2), variance numeric(14,2));
CREATE TABLE reporting.pricing_review_fact (review_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, kind
    text, sku_id uuid, raised_at timestampz, resolved_at timestampz);
CREATE TABLE reporting.fleet_health (device_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, location_id
    uuid, last_seen_at timestampz, app_version text,
    snapshot_version bigint, snapshot_age_hours int, pending_events int, oldest_pending_age_min int, battery_pct
    smallint, storage_free_mb int, peripherals jsonb, floor_state text);
CREATE TABLE reporting.exception_item (audit_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, location_id
    uuid, event_type_code text NOT NULL, severity text NOT NULL,
    occurred_at timestampz NOT NULL, reviewer_role_template uuid, subject_table text, subject_id uuid,
    document_id uuid, actor_user_id uuid,
    acknowledged_at timestampz, acknowledged_by uuid, disposition text, escalated_at timestampz);

CREATE TABLE reporting.report_definition (report_id varchar(40) PRIMARY KEY, name_en text NOT NULL, name_si
    text, name_ta text, decision_supported text NOT NULL,
    audience_roles text[] NOT NULL, scope_class text NOT NULL CHECK (scope_class IN
    ('OWN', 'FEDERATION_VIEW', 'PARTY')), parameters_schema jsonb NOT NULL,
    source_kind text NOT NULL CHECK (source_kind IN ('PROJECTION_QUERY', 'SQL_TEMPLATE')), source text NOT NULL,
    a4_template text, xlsx_layout jsonb,
    freshness_rule text NOT NULL DEFAULT 'MIN_LOCATIONS', status text NOT NULL DEFAULT 'DRAFT', version int NOT
    NULL DEFAULT 1, owner_entity_id uuid NOT NULL);
CREATE TABLE reporting.report_run (run_id uuid PRIMARY KEY, report_id varchar(40) NOT NULL, requested_by uuid
    NOT NULL, owner_entity_id uuid NOT NULL,
    scope jsonb NOT NULL, parameters jsonb NOT NULL, language text NOT NULL, freshness timestampz,
    stale_locations int, status text NOT NULL DEFAULT 'REQUESTED',
    object_key text, requested_at timestampz NOT NULL DEFAULT now(), rendered_at timestampz, error text);
CREATE TABLE reporting.export_job (job_id uuid PRIMARY KEY, source text NOT NULL, requested_by uuid NOT NULL,
    owner_entity_id uuid NOT NULL, scope jsonb NOT NULL,
    parameters jsonb NOT NULL, status text NOT NULL DEFAULT 'QUEUED', object_key text, row_count int, expires_at
    timestampz, error text);
CREATE TABLE reporting.dashboard_tile (tile_id varchar(40) PRIMARY KEY, name_en text, name_si text, name_ta
    text, role_templates text[], source text NOT NULL, drill_report_id varchar(40), position smallint);
CREATE TABLE reporting.user_dashboard (user_id uuid NOT NULL, tile_id varchar(40) NOT NULL, position smallint,
    PRIMARY KEY (user_id, tile_id));

-- analyst role (replica): CREATE ROLE app_analyst NOLOGIN; GRANT USAGE ON SCHEMA reporting TO app_analyst;
-- GRANT SELECT ON ALL TABLES IN SCHEMA reporting TO app_analyst;
-- login role per analyst inherits app_analyst; the pooler's connect hook sets app.scope_class =
    'FEDERATION_VIEW' (19A 1) and audits the session.

```

3.1 Seed data

```

# seed/m8/report-definitions.yaml (twelve phase-1 placeholders; excerpt)
- id: daily-sales-by-shop decision: "What sold yesterday and how much cash goes to the bank" audience:
    [shop-in-charge, supervisor, mpcs-accountant]
  scope: OWN params: {locationId: uuid, date: date} source: PROJECTION_QUERY sales_daily+cash_daily a4:
    daily_sales.html xlsx: [date, sku, qty, net, tax]
- id: sales-by-sku-tag decision: "What to reorder and what to drop" audience: [mpcs-stores,
    mpcs-manager] scope: OWN params: {entityId, from, to, tag?}
- id: till-sessions-variance decision: "Is cash under control" audience: [supervisor, mpcs-accountant]
  scope: OWN
- id: stock-position-expiry decision: "What to count, mark down or move" audience: [shop-in-charge,
    mpcs-stores] scope: OWN
- id: loss-by-category decision: "Where losses arise" audience: [mpcs-accountant, federation-view]
  scope: OWN|FEDERATION_VIEW
- id: shrinkage-pct decision: "Which shops need attention" audience: [mpcs-manager,
    federation-view]
- id: dispute-register decision: "What is unresolved with our counterparties" audience: [seller-sales,
    buyer-stores] scope: PARTY
- id: discount-markdown-cost decision: "What promotions and markdowns cost us" audience: [mpcs-pricing]
- id: vat-summary decision: "What to file" audience: [accounts]
- id: exposure-statements decision: "Who to chase" audience: [seller-accounts] scope:
    OWN|PARTY(statement)
- id: receivables-ageing decision: "Khata risk" audience: [mpcs-accountant]
- id: fill-rate-delivery decision: "Are we meeting service levels" audience: [seller-sales,
    federation-view]
- id: fleet-sync-health decision: "Which shops are dark" audience: [devops, federation-view]
# seed/m8/dashboard-tiles.yaml: yesterday-sales, cash-to-bank, variance, expiring-7d, negative-lots,
    open-disputes, exposure-utilisation, fleet-dark (per role template)
# seed/m8/permissions.yaml (appended): rpt.report.run (ENTITY; audience checked per definition) |

```

```

    rpt.export.run (ENTITY) | rpt.definition.manage (F, mfa on activate)
#   sys.projection.rebuild (F, mfa) | rpt.analyst.query (F)

```

4. Package layout

```

lk.coopfed.knoweb.m8reporting/  @ApplicationModule(allowedDependencies =
    {"kernel","m1party::query","m2catalogue::query"})  -- read side: no layer-2 api calls
api/      commands (6), events (5 families)
query/    ReportingQueries (dashboard, reportData, freshness, exceptionQueue, projectionStatus, named
    projections for M4/M5/M7 screens)
internal/
projection/ Projection<E> base (upsertFrom(event), rebuildFrom(archive)), thirteen consumers,
    ProjectionStateService, RebuildService (shadow table swap)
freshness/ FreshnessConsumer (sync.batch_received.v1), FreshnessResolver (min over locations, stale
    count)
eod/      EndOfDayConsumer (location.day_closed.v1), RollupJob (monthly, FY), LateEventHandler (updates
    closed days)
catalogue/ ReportDefinition handlers, DefinitionValidator (fixture render), CatalogueQueries
render/    ReportRunWorker (A4Renderer), ExportWorker (xlsx via a streaming writer), ObjectStore keys,
    ExpiryJob
dashboard/ TileService
exception/ ExceptionItemConsumer (audit REVIEW/ALERT + AUDIT_REVIEWED), EscalationJob
analyst/   AnalystRoleService (create/revoke login roles on the replica through a privileged job),
    SessionAudit
web/      DashboardController, CatalogueController, RunController, ExportController, ExceptionController,
    FreshnessController, DefinitionController, AnalystController

```

5. API slice: openapi/m8reporting.yaml (excerpt)

```

/v1/reporting/dashboard      get (scope) -> { tiles: [{tileId, value, delta?, freshness,
    staleLocations, drillReportId}] }
/v1/reporting/definitions   get (audience)  post register (F)  /{id}/activate | /retire post
/v1/reporting/reports/{id}/data  get (scope, params) -> { columns, rows, freshness, staleLocations,
    generatedAt }
/v1/reporting/reports/{id}/runs  post requestRun {scope, params, language} -> {runId}  get list
/v1/reporting/runs/{runId}      get -> {status, objectKey?, freshness}
/v1/reporting/exports         post requestExport {source, scope, params} -> {jobId}  /{jobId}
    get
/v1/reporting/freshness       get (entityId | locationId) -> {appliedThrough, staleLocations[],
    devicesReporting}
/v1/reporting/exceptions      get (scope, severity, since, acknowledged?)
/v1/reporting/projections     get status  /{name}/rebuild post {fromSeq} (F, mfa)
/v1/reporting/analysts        post issue {userId} | delete revoke
# example - dashboard tile with freshness
{ "tileId": "yesterday-sales", "value": 184250.00, "delta": 0.06, "freshness": "2026-10-14T06:10:00+05:30",
  "staleLocations": 1, "drillReportId": "daily-sales-by-shop" }

```

6. The projection pattern, freshness and end-of-day

```

// projection/Projection.kt - every consumer is an instance of this pattern
abstract class Projection<E : DomainEvent>(val name: String, val types: Set<String>) {
    abstract fun upsert(e: E, tx: Tx) // idempotent by aggregate id (+ event id for facts)
    @EventConsumer(types = types, consumer = "m8.$name") // kernel inbox guarantees once-per-event
    fun on(e: E) = tx { upsert(e, it); state.advance(name, e.sourceSeq, e.receivedAt) }
    fun rebuild(fromSeq: Long) { // RebuildService
        state.mark(name, REBUILDING, shadow = "${name}_shadow"); createShadowLike(name)
        archive.replay(types, fromSeq) { e -> tx { upsertInto(shadow, e, it) } } // reads kernel.event_outbox
        partitions in (source, seq) order
        tx { swap(name, shadow); state.mark(name, LIVE, lastSeq = archive.headSeq()) } // ALTER TABLE ... RENAME
        in one transaction; consumer resumes
    }
}
// receipt_line_fact.upsert: INSERT ... ON CONFLICT (received_at, line_id) DO NOTHING; voided -> UPDATE
// receipt_fact.voided_by (fact rows are never deleted)
// stock_position.upsert: INSERT ... ON CONFLICT (location, batch, condition) DO UPDATE SET qty_on_hand =
//   qty_on_hand + delta, ... (delta applied once thanks to the inbox)
// canonical_sku_id = catalogue.resolveAlias(e.skuId) at projection time; a later merge event re-points
//   historical facts by a corrective job (sku.merged.v1 consumer)

// freshness/FreshnessConsumer: on sync.batch_received.v1 -> location_freshness.applied_through =
//   max(existing, batch.lastEventReceivedAt); devices_reporting
// FreshnessResolver.forLocations(ids): { min(applied_through), count(applied_through < now - staleThreshold)
// }
// sales_daily.freshness = FreshnessResolver.forLocations([location]).min at the time of the last upsert
//   (re-stamped by LateEventHandler)

// eod/EndOfDayConsumer: on location.day_closed.v1(location, businessDate) ->
//   recompute sales_daily and cash_daily rows for (location, businessDate) from facts (cheap: one location,
//   one day); stamp freshness;
//   publish end_of_day.completed.v1 with totals. LateEventHandler: any fact for a closed (location, day)
//   triggers the same recompute.
// RollupJob (monthly): sales_monthly by entity/district/tag from sales_daily; FY roll-ups by
//   entity.financial_year_start_month (M1)

```

Aspect	Design
Lag detection	projection_state.last_event_at versus the archive head; over projection_lag_alert_seconds → LAGGING and an ALERT; the dashboard shows the lag beside the freshness stamp.
Cost columns	Facts carry unit_cost for OWN and FEDERATION_VIEW; PARTY-readable projections (trade_document_fact, dispute_register) have no cost columns at all, so masking is structural.

7. Catalogue, rendering, export, dashboards, analyst role

```

// catalogue/DefinitionValidator: source_kind PROJECTION_QUERY -> named query over reporting.* only
//   (allow-list of tables); SQL_TEMPLATE -> parsed, must reference
//   reporting.* only, parameters bound (no string interpolation), EXPLAIN-checked against the replica;
//   a4_template must render on fixture rows in three languages
// render/ReportRunWorker (@EventConsumer report.requested.v1): data = ReportingQueries.reportData(def, scope,
//   params) under the requester's RLS session;
//   html = template.render(data, freshness, language); pdf = a4.render(html); store; run READY; publish
//   report.rendered.v1
// render/ExportWorker: streams rows to xlsx (SXSSF-style streaming) with the layout; over the cap -> paginate

```

```

    sheets; scope banner and freshness in the first rows
// dashboard/TileService: tile.source is a named projection query; cached 60 s per (tile, scope); freshness
    from FreshnessResolver for the scope's locations
// exception/ExceptionItemConsumer: audit events with severity REVIEW/ALERT -> exception_item; AUDIT_REVIEWED
    -> acknowledged_*; EscalationJob (hourly): ALERT
//    unacknowledged > 3 days -> escalated_at + exception_report.generated.v1 to the next reviewer role
// analyst/AnalystRoleService.issue(userId): privileged job on the replica: CREATE ROLE analyst_<user> LOGIN
    IN ROLE app_analyst PASSWORD <one-time>;
//    ALTER ROLE ... SET app.scope_class = 'FEDERATION_VIEW'; audit ANALYST_ROLE_ISSUED; revoke drops the role;
    pooler routes analyst logins to the replica only

```

8. Screens

Dashboard (web, MPCs manager)

```

+-----+
| Kandy South MPCs (M042) . 12 shops . Data: shops synced up to 06:10 today; 1 shop not yet synced |
| [Yesterday sales Rs 1,842,500 (+6%)] [Cash to bank Rs 1,610,200] [Variance Rs -1,250 (2 sessions)] |
| [Expiring 7d: 38 batches] [Negative lots: 4] [Open disputes: 2] [Exposure 92% of limit !] [Dark shops: 1] |
| each tile: value . small freshness line . tap -> report |
+-----+
Report viewer: header with decision and audience; parameters form; freshness banner ("includes shops synced up
to ..; 1 shop excluded: Shop 07");
tabs Data | Print (A4 preview) | Export; run history

```

Screen	Components	Message ids (m8.)	Notes
Dashboard	tiles grid, FreshnessStamp, drill links	dashboard.*	Tiles per role template; user may pin/order
Catalogue	list grouped by decision; parameters forms from schema	catalogue.*	Audience-filtered
Report viewer	data table, A4 preview, export button, run history, FreshnessStamp	report.*	–
Exception queue	shared ExceptionQueue component; dispositions call the kernel command	exception.*	–
Freshness and fleet	map or list of locations by applied_through; stuck devices; projection lag	freshness.*, fleet.*	Federation, DevOps
Definition editor	form with fixture preview in three languages; activation with StepUp	definition.*	F only
Analyst access	issue/revoke; session audit list	analyst.*	F admin

9. Tests

Layer	Cases	Target
Rebuild equivalence (first test)	For random event sequences from the 21A–27A fixtures: live projection rows equal a from-zero rebuild for all thirteen projections	Row-for-row equality
Idempotency	Replay any event twice: no change	All consumers
Unit	Alias resolution and merge re-pointing; freshness min and stale count; end-of-day totals to the cent; ageing and shrinkage formulas; late-event recompute; escalation ageing	All branches
Definition validation	SQL template referencing an operational schema refused; parameters unbound refused; template render in three languages	All cases
RLS matrix	Shop, MPCs, distributor (PARTY: no cost columns exist), Federation view, analyst role (reporting only; operational grant absent)	Full matrix
Contract	Every operation; report data equals rendered data; export respects scope; dashboard tile freshness	Every operation
Scenario	Flows 6.1–6.7 of doc 28: late sync after two days; wrong consumer fixed by rebuild; analyst session	Seven flows
Performance	1,000 events/s projection throughput on the two-instance stack; end-of-day for 10,000 locations under 120 min; dashboard p95 under 2 s on the replica; export of 500,000 rows under 10 min	Four targets
Web	Playwright: dashboard drill → report run → A4 → export; register a definition with fixture preview	Two flows

10. Tickets in build order

#	Ticket	Est. (days)	Done when
M8-01	Scaffold; migration with RLS; projection base, ProjectionStateService, RebuildService with shadow swap; equivalence test harness	3	Harness runs on a sample projection
M8-02	Receipt, receipt line, sales_daily, cash_daily, till_session projections; alias resolution	3	Equivalence and idempotency green
M8-03	Stock position, stock movement, loss projections; drift comparison job against M5	2	As above
M8-04	Trade document, dispute register, exposure, customer receivables, pricing review, fleet health, exception item projections	3	As above
M8-05	Freshness consumer and resolver; end-of-day consumer; late-event handler; roll-up job	2	Flow 6.4
M8-06	Catalogue: definitions as data, validator, seed of twelve; run worker with A4; export worker; expiry job	2.5	Twelve definitions render in three languages
M8-07	Dashboards, tiles, user pins; exception queue queries; escalation job	1.5	Flow 6.2, 6.7
M8-08	Analyst role service on the replica; session audit; pooler routing	1	Flow 6.6
M8-09	Web screens (seven) and Playwright flows	2.5	Two flows green
M8-10	Contract freeze: m8reporting.yaml v1; review with M9 and doc 31; workshop hand-off format agreed	0.5	M9 acknowledges

Total about 21 developer-days.

11. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Workshop timing and phase-1 set (doc 28 DR-1, DR-2)	Twelve placeholders as seeded; definitions replaced as data	Federation	Seed file changes only
Analyst role issuance, stale threshold, export cap (DR-3 to DR-5)	As proposed	Federation, DevOps	Config
Reporting schema excluded from doc 18 invariants (DR-6)	Adopted; CR-28-1	Architecture review	–
Replica availability in development	Compose profile adds a streaming replica; tests run against it	Platform pair	Run against the primary with a flag

Point	Assume	Decider	If different
Customer names on receivables reports	Resolved at render time under M7's RLS; never stored in projections	Federation legal	–
Cost of goods sold basis in sales_daily	Entity average cost at issue from the receipt line (M5's unit_cost_at_movement carried into the bundle by M6)	M5, M6 developers	Join to movement facts instead

End of document. 28A v0.1; m8reporting/README.md, the definition seed and the equivalence harness become the living version once merged.